Week 11 Capstone Project

AI Commerce Intelligence Platform: Recommendations, Demand Forecasting & Shopping Assistant

Recommender Systems • Time-Series Forecasting • Hybrid Search • Agentic Shopping Assistant • Review Intelligence • A/B Testing • Real-time Serving

Duration: 5 Days

📌 Scenario

Your new client is ShopKaro (fictional), a Pakistani e-commerce marketplace with 50,000 products (fashion, electronics, home, beauty, groceries) and 1 million monthly visitors. Growth has slowed, and management has identified four problems:

- Every user sees the same homepage. A university student and a mother of three get the same “Top Products”, so conversion is low
- Search is keyword-only. A user types "eid ke liye lawn suit under 5000" or "bachon ke liye school bag" and gets poor results
- Inventory planning is guesswork. Popular items run out before Eid, Ramadan and 11.11 sales, while slow items fill the warehouse
- Thousands of reviews arrive every week in English, Urdu and Roman Urdu, and nobody reads them

Your task is to build a production-ready AI Commerce Intelligence Platform: a personalised recommendation engine, a demand forecasting system that drives inventory decisions, a smart search and UrduLish shopping assistant, and review intelligence — all served in real time and proven with A/B testing.

Previous weeks covered LLM agents, tabular ML, multi-agent systems and fine-tuning. This week adds two of the most valuable ML skills in industry: recommender systems and time-series forecasting, combined with an LLM assistant on top.

The final product should be deployable for an actual client.

Technologies Expected

The project should expose students to a complete modern commerce-AI stack.

Area	Tools
Data	Pandas / Polars, DuckDB, PostgreSQL
Recommenders	implicit (ALS / BPR), LightFM, Surprise, PyTorch two-tower models, RecBole (optional)
Embeddings	Sentence-Transformers, CLIP (for product images, optional), multilingual-e5 / BGE-M3
Search	BM25 (rank_bm25 / Elasticsearch / OpenSearch), FAISS / Qdrant, cross-encoder rerankers
Forecasting	statsmodels, Prophet, StatsForecast, LightGBM, NeuralForecast / Darts (N-BEATS, TFT)
NLP for Reviews	Multilingual transformers, LLM-based aspect extraction, BERTopic

Area	Tools
LLM & Agents	OpenAI GPT, Claude, Gemini; LangGraph / function calling
Experiment Tracking	MLflow or Weights & Biases
Serving & Caching	FastAPI, Redis
Experimentation	SciPy / statsmodels for A/B testing
Frontend	Streamlit or Next.js
Deployment	Docker, Railway, Render, AWS

Project Objectives

Build a production-ready commerce AI platform capable of:

- ✅ Customer segmentation and behaviour analysis
- ✅ Personalised “Recommended for you” lists
- ✅ “Similar products” and “Frequently bought together”
- ✅ Handling cold start for new users and new products
- ✅ Product demand forecasting with seasonality (Eid, Ramadan, 11.11, school season)
- ✅ Reorder points, safety stock and stock-out alerts
- ✅ Hybrid semantic + keyword search in English, Urdu and Roman Urdu
- ✅ UrduLish shopping assistant that uses real tools (search, recommend, cart, order status)
- ✅ Review sentiment and aspect analysis
- ✅ Offline evaluation with ranking and forecasting metrics
- ✅ A/B test design and statistical analysis
- ✅ Real-time serving with caching
- ✅ Dockerized deployment

Datasets

Use public e-commerce datasets and adapt them to the Pakistani context (PKR prices, local categories, local festivals). Document every source.

Interactions: user events (view, add-to-cart, purchase) — minimum 500,000 events. Suggested: RetailRocket, Olist (Brazilian E-Commerce), H&M Personalized Fashion, Instacart, or Amazon Reviews.

Product Catalogue: product id, title, description, category, brand, price, image (optional), stock.

Sales History: daily sales per product for at least 2 years. Students must add festival effects (Eid ul Fitr, Eid ul Adha, Ramadan, 11.11, Black Friday, back-to-school) to the calendar.

Reviews: at least 5,000 reviews. Include synthetic Roman Urdu / UrduLish reviews if needed (e.g. "Kapra acha hai lekin delivery bohot late thi").

Week 11 — Day 1: Data Foundation, Customer Analytics & Segmentation

> 📌 **Scenario:** Before recommending anything, understand the customers. Who buys what, how often, and when? Today you turn millions of raw clicks into clean, useful data and real business insight.

Task 1 — Data Pipeline

Load all datasets into DuckDB or PostgreSQL and implement:

- Event cleaning (bots, duplicate events, sessions with 1,000+ clicks)
- Session building (30-minute inactivity rule)
- Mapping to Pakistani context (PKR prices, local categories, festival calendar)
- User and item ID mapping

Draw the data model (ER diagram).

Task 2 — Exploratory Analysis

Create visualisations and write one-line business insights for:

- Conversion funnel: view → cart → purchase
- Top categories and brands
- Long-tail: what % of products get 80% of sales?
- Sales by day of week, hour and festival period
- Data sparsity of the user–item matrix

Task 3 — Customer Segmentation

Implement:

- RFM analysis (Recency, Frequency, Monetary)
- K-Means or HDBSCAN clustering on behaviour features
- Name and describe each segment (e.g. “Eid Shoppers”, “Bargain Hunters”, “Loyal Electronics Buyers”)
- Recommend one marketing action per segment

Task 4 — Cohort & Retention Analysis

Build:

- Monthly cohort retention heatmap
- Repeat purchase rate
- Customer Lifetime Value (simple CLV estimate)
- Churn definition (e.g. no purchase in 90 days)

Task 5 — Feature Engineering & Splits

Prepare features for later days:

- User features (segment, avg order value, favourite categories)
- Item features (category, price band, popularity, text embedding of title + description)
- Time-based train / validation / test split — never random split for recommendations (explain why)
- A cold-start test set: users and items not seen in training

---

Week 11 — Day 2: Recommendation Engine

> 📌 **Scenario:** Amazon says 35% of its revenue comes from recommendations. Today you build ShopKaro’s engine: “Recommended for you”, “Similar products” and “Frequently bought together”.

Task 1 — Baselines

Implement and evaluate:

- Most popular overall
- Most popular in the user's favourite category
- Recently trending (last 7 days)

Any advanced model must beat these.

Task 2 — Collaborative Filtering

Implement:

- Item-to-item similarity (co-occurrence / cosine)
- Matrix factorisation for implicit feedback (ALS or BPR with the implicit library)
- Hyperparameter tuning (factors, regularisation, iterations)

Explain the difference between explicit and implicit feedback.

Task 3 — Content-Based & Hybrid Models

Implement:

- Content-based similarity using text embeddings of product titles and descriptions
- Hybrid model with LightFM (interactions + item / user features) or a PyTorch two-tower model
- Cold-start strategy for new users (segment / onboarding questions) and new items (content embeddings)

Task 4 — Offline Evaluation

Measure every model on the time-based test set:

- Precision@10, Recall@10
- NDCG@10 and MAP@10
- Hit Rate
- Catalogue coverage
- Diversity and novelty
- Performance on cold-start users and items

Create a model comparison table and choose the production model.

Task 5 — Business Rules & Re-ranking

Apply a re-ranking layer on top of the model:

- Remove out-of-stock and already-purchased items
- Diversity (not 10 items from the same brand)
- Boost items on sale or with high margin (configurable)
- Festival boost (e.g. lawn suits before Eid)
- "Frequently bought together" using association rules (Apriori / FP-Growth)

Explain each recommendation in one line, e.g. "Kyun ke aap ne Samsung Galaxy dekha, yeh covers aur chargers bhi pasand kiye ja rahe hain."

Week 11 — Day 3: Demand Forecasting & Inventory Intelligence

> 📌 **Scenario:** Last Eid, ShopKaro ran out of its top 200 lawn suits in 3 days, while 10,000 winter jackets sat unsold in the warehouse. Today you build a forecasting system that tells the operations team what to stock, how much, and when.

Task 1 — Time-Series Analysis

For top categories and products:

- Trend, seasonality and residual decomposition
- Weekly and yearly seasonality
- Festival effects (Eid, Ramadan, 11.11) — note that Islamic festival dates move every year
- Stationarity tests (ADF)
- Intermittent demand (products that sell rarely)

Task 2 — Forecasting Models

Implement and compare:

- Naive and seasonal naive baselines
- ARIMA / ETS (StatsForecast or statsmodels)
- Prophet with custom festival holidays
- LightGBM with lag, rolling and calendar features (global model across products)
- Optional: N-BEATS or Temporal Fusion Transformer (NeuralForecast / Darts)

Task 3 — Backtesting & Evaluation

Implement rolling-origin backtesting (at least 4 folds) and measure:

- MAE, RMSE
- MAPE and sMAPE (and why MAPE breaks for low-volume items)
- WAPE (recommended for retail)
- Accuracy during festival periods separately

Report results by category and by sales volume (fast vs slow movers).

Task 4 — Probabilistic Forecasts

A single number is not enough for inventory. Produce:

- Prediction intervals (P10, P50, P90) using quantile regression or conformal prediction
- Calibration check: does the P90 forecast cover reality ~90% of the time?

Task 5 — Inventory Decisions

Convert forecasts into actions:

- Safety stock and reorder point for each product
- Stock-out risk alerts for the next 14 days
- Overstock alerts (slow movers to discount)
- Pre-festival stocking plan for the top 100 products

Simulate last year: how many stock-outs and how much dead stock would your system have prevented?

Week 11 — Day 4: Smart Search, Shopping Assistant & Review Intelligence

> 📌 **Scenario:** Customers do not think in keywords. They say "shaadi ke liye red lehenga 20 hazaar tak" or "laptop for coding under 1.5 lac". Today search becomes intelligent, an AI assistant starts helping customers shop, and reviews become insights.

Task 1 — Hybrid Search

Build product search with:

- BM25 keyword search
- Semantic vector search (multilingual embeddings)
- Hybrid scoring (e.g. reciprocal rank fusion)
- Cross-encoder reranking for the top 50 results
- Query understanding: extract category, price range, colour and occasion from the query (LLM or rules)
- Spelling and Roman Urdu variations ("joota" / "juta" / "shoes")

Evaluate with 50 labelled queries using NDCG@10 and compare keyword vs semantic vs hybrid.

Task 2 — Personalised Search

Re-rank search results using the user's profile from Day 2:

- Same query, different users → different order
- Measure the change in NDCG on users with history

Task 3 — UrduLish Shopping Assistant

Build an LLM agent with tools:

- search_products
- get_recommendations
- compare_products
- add_to_cart
- get_order_status
- get_review_summary

The assistant must never invent a product, price or stock level — every fact must come from a tool.

Example: Customer: "Ammi ke liye Eid gift chahiye, 3000 tak." Assistant: "Ji zaroor! Yeh 3 options aap ke budget mein hain — ek khoobsurat embroidered shawl (2,450), ek perfume gift set (2,800)…"

Task 4 — Review Intelligence

Analyse reviews in English, Urdu and Roman Urdu:

- Sentiment classification (fine-tuned multilingual model or LLM)
- Aspect-based sentiment: quality, price, delivery, size / fit, packaging, seller behaviour
- Topic discovery with BERTopic
- Fake / spam review detection (duplicates, burst patterns, extreme ratings)
- Auto-generated "What customers say" summary per product

Example: "Zyada tar customers ko kapre ki quality pasand aayi, lekin 30% ne kaha size chhota hai."

Task 5 — Seller & Management Insights

Build insights that management can act on:

- Top complaints by category and seller
- Products with falling sentiment over time

- Sellers with delivery problems
- Weekly automated insight report (LLM-written, numbers from SQL)

Week 11 — Day 5 (Capstone): Real-time Serving, A/B Testing, Deployment & Client Handover

> 📌 **Scenario:** Offline metrics do not pay salaries — revenue does. Today you serve everything in real time, prove the impact with an A/B test, and present it to ShopKaro's CEO.

Task 1 — Real-time Serving

Build FastAPI endpoints:

- GET /recommendations/{user_id}
- GET /similar/{product_id}
- GET /bought-together/{product_id}
- GET /search?q=…
- POST /assistant/chat
- GET /forecast/{product_id} and GET /inventory/alerts
- GET /reviews/summary/{product_id}

Use batch pre-computation + Redis caching for recommendations and update with recent session clicks in real time. Target: under 100 ms for recommendations.

Task 2 — A/B Testing

Design and analyse an experiment:

- Hypothesis, primary metric (conversion rate / revenue per visitor) and guardrail metrics
- Sample size and test duration calculation (power analysis)
- Simulate traffic: control (popular items) vs treatment (your recommender)
- Statistical significance (z-test / t-test), confidence intervals
- Common mistakes: peeking, novelty effect, multiple testing

Write a one-page result: ship or don't ship?

Task 3 — Dashboard & Deployment

Build a dashboard with:

- Customer view: personalised homepage, search, assistant chat
- Operations view: forecasts, stock-out and overstock alerts
- Management view: segments, review insights, A/B test results

Deploy with Docker (API, Redis, database, dashboard), environment variables, health checks, logging and basic CI. Define a retraining schedule (daily recommendations, weekly forecasts).

Task 4 — Stakeholder Demonstration

Prepare a 10-minute live demo showing:

- Two different users seeing different homepages
- A new user (cold start) getting sensible recommendations
- UrduLish search: "eid ke liye lawn suit under 5000"
- Shopping assistant finding and comparing products
- Review summary for a product

- Demand forecast with intervals and a stock-out alert before Eid
- A/B test results and estimated revenue uplift
- Business impact estimate (conversion, stock-outs prevented, dead stock reduced)

Task 5 — Documentation & Future Enhancements

Write:

- System architecture diagram
- Model cards for the recommender, forecaster and review models
- API documentation
- User guides for marketing, operations and management teams
- Monitoring and retraining guide

Propose future enhancements such as: image-based “shop the look” search, dynamic pricing, real-time streaming with Kafka, push notification personalisation, WhatsApp commerce, delivery-time prediction, and reinforcement-learning-based recommendations.

Evaluation Rubric

Criteria	Weight
Data pipeline, analytics & segmentation	10%
Recommendation engine & offline evaluation	20%
Demand forecasting & inventory decisions	20%
Hybrid search & shopping assistant	15%
Review intelligence	10%
Real-time serving, A/B testing & deployment	15%
Documentation & demo	10%

Minimum targets: Recommender beats the popularity baseline on NDCG@10 by at least 20%; forecasting model beats seasonal naive on WAPE; hybrid search beats keyword search on NDCG@10; recommendation API under 100 ms (cached); assistant invents zero products or prices.

Final Submission Requirements

The final submission should include:

- ✅ Clean data pipeline and data model
- ✅ EDA, cohort analysis and customer segmentation report
- ✅ Recommendation engine (baselines, collaborative filtering, content-based, hybrid)
- ✅ Cold-start strategy and re-ranking layer
- ✅ Offline evaluation report with ranking metrics
- ✅ Demand forecasting models with backtesting results
- ✅ Probabilistic forecasts and inventory recommendations
- ✅ Hybrid search with evaluation
- ✅ UrduLish shopping assistant with tool calling
- ✅ Review sentiment, aspect analysis and summaries

- ✅ A/B test design and analysis report
- ✅ FastAPI backend with Redis caching
- ✅ Customer, operations and management dashboards
- ✅ Dockerized deployment with a live link
- ✅ GitHub repository with README, setup guide, API docs and architecture diagram
- ✅ 10-minute demo video and slide deck

The finished project should resemble a commercial AI platform that an e-commerce company could deploy to increase conversion, reduce stock-outs and dead stock, and understand its customers — demonstrating modern best practices in recommender systems, time-series forecasting, search, LLM agents, experimentation and production ML engineering.